

# **PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ**

*FACULTAD DE CIENCIAS E INGENIERÍA*

*ESPECIALIDAD DE INGENIERÍA INFORMÁTICA*



Proyecto de Diseño y Desarrollo de Software [1INF54-0982]

## **Estándar de Pruebas de Software**

**Equipo: 6F**

### **Integrantes:**

- Gabriel Angel Quispe Cerron
- Roly Alvaro Huacre Samaniego
- Amira Paola Rodriguez Almora
- Diego Alfredo Carmona Chumbimuni

2026-2

Lima, 15 de septiembre

## Historial de versiones

| Fecha      | Versión | Descripción                                                                        | Autor     |
|------------|---------|------------------------------------------------------------------------------------|-----------|
| 15/09/2026 | 1.0     | Se ha redactado la primera versión de los estándares de programación web frontend. | Equipo 6F |

# Contenido

|                                                 |          |
|-------------------------------------------------|----------|
| <b>1. Introducción</b>                          | <b>4</b> |
| <b>2. Convenciones de nombrado de pruebas</b>   | <b>4</b> |
| <b>3. Uso de mocks</b>                          | <b>5</b> |
| 3.1 Backend                                     | 5        |
| 3.2 Frontend                                    | 5        |
| <b>4. Umbral mínimo de cobertura de código</b>  | <b>5</b> |
| <b>5. Organización de pruebas</b>               | <b>6</b> |
| <b>6. Pruebas de Extremo a Extremo (E2E)</b>    | <b>6</b> |
| <b>7. Aislamiento de Estado en Integración</b>  | <b>6</b> |
| <b>8. Pruebas de Rendimiento y Benchmarking</b> | <b>6</b> |
| <b>9. Referencias y recursos</b>                | <b>7</b> |

# 1. Introducción

Este documento define las normativas y buenas prácticas de pruebas de software aplicables al backend (Java / Spring Boot) y al frontend (React / TypeScript) del sistema SysMile. Incluye directrices para los módulos de optimización de rutas (ALNS / IPSO), donde la precisión de los cálculos es crítica para garantizar la calidad del servicio.

## 2. Convenciones de nombrado de pruebas

Se adopta la convención **given\_when\_then** para estructurar las pruebas en el backend, así como su equivalente en las descripciones de los bloques *it()* y *test()* en el frontend, con el fin de que cada caso documente claramente el escenario evaluado.

```
Java
@Test
void
givenEmptyRoute_whenCalculatingEta_thenThrowsEmptyRouteException(
) {
    // arrange
    Route route = RouteTestFactory.withNoStops();

    // act & assert
    assertThrows(EmptyRouteException.class, () ->
etaCalculator.calculate(route));
}
```

- El nombre del método debe seguir la estructura *given{Contexto}\_when{Acción}\_then{ResultadoEsperado}* utilizando la convención camelCase.
- Estructurar el cuerpo de cada prueba en las secciones **Arrange**, **Act** y **Assert** (o Given, When, Then), separadas por saltos de línea o comentarios explicativos.
- En el frontend, utilizar *describe()* para agrupar las pruebas por componente o función, e *it('debería ... cuando ...')* para especificar cada caso de prueba.

```
TypeScript
describe('RouteMap', () => {
    it('debería mostrar el marcador en rojo cuando el vehículo
está retrasado', () => {
        render(<RouteMap vehicles={[delayedVehicle]} />);
    });
});
```

```
expect(screen.getByTestId('vehicle-marker')).toHaveClass('vehicle
-marker--danger');
});
});
```

## 3. Uso de mocks

### 3.1 Backend

- Usar Mockito para simular dependencias externas (repositorios, clientes HTTP de geocodificación, colas de mensajería).
- Simular condiciones de fallo (timeouts, vehículos fuera de servicio) mediante mocks que arrojen excepciones controladas, verificando su correcta traducción a excepciones de dominio.
- No emplear mocks en las clases matemáticas puras de los algoritmos ALNS / IPSO; estas deben validarse con datos reales de entrada y salida.
- Las pruebas de integración deben utilizar Testcontainers con PostgreSQL para evitar simular la capa de persistencia.

### 3.2 Frontend

- Usar Mock Service Worker (MSW) para interceptar solicitudes a la API durante las pruebas de componentes, evaluando escenarios de éxito, error y latencia.
- Simular pérdida de conexión mediante eventos controlados en el cliente WebSocket para verificar las reacciones de la interfaz de usuario.
- Evitar el uso de mocks en funciones utilitarias internas (formateo de fechas, cálculos geométricos); estas deben evaluarse con datos de prueba directos.

## 4. Umbral mínimo de cobertura de código

| Módulo                              | Cobertura mínima (líneas) | Cobertura mínima (ramas) |
|-------------------------------------|---------------------------|--------------------------|
| Backend — General                   | 80%                       | 70%                      |
| Backend — Autenticación y seguridad | 90%                       | 85%                      |
| Backend — Algoritmos ALNS / IPSO    | 90%                       | 85%                      |

|                                                    |     |     |
|----------------------------------------------------|-----|-----|
| Frontend — General                                 | 75% | 65% |
| Frontend — Visualizador de mapa (lógica de estado) | 85% | 75% |

- La cobertura se mide con JaCoCo en el backend y con Vitest en el frontend, integrados en el flujo de CI.
- Ningún Pull Request podrá integrarse a la rama principal si reduce la cobertura por debajo del umbral mínimo configurado.
- Los módulos de optimización exigen pruebas explícitas de casos límite (instancias sin pedidos, capacidad agotada) y pruebas de regresión numérica.

## 5. Organización de pruebas

- Backend: ubicar las pruebas en `src/test/java` replicando la estructura de paquetes de la clase probada, usando los sufijos *Test* para unitarias e *IT* para integración.
- Frontend: ubicar los archivos de prueba junto al componente objetivo con la extensión `.test.tsx` o dentro de un directorio `__tests__` adyacente.
- Pruebas E2E: organizar en un directorio `e2e/` independiente, agrupadas por flujos de usuario.
- Fixtures y datos compartidos: centralizar en directorios dedicados para evitar duplicidad entre suites de prueba.

## 6. Pruebas de Extremo a Extremo (E2E)

- Para las pruebas automatizadas de extremo a extremo, que simulan la interacción real del usuario con el navegador, se utilizará el framework Playwright (o alternativamente Cypress).
- Dado su costo de ejecución y mantenimiento, las pruebas E2E deben limitarse exclusivamente a los "Caminos Felices" (Happy Paths) y flujos críticos de negocio (ej. iniciar sesión exitosamente, visualizar el mapa de rutas, y asignar un vehículo). Las validaciones de campos y errores de borde deben delegarse a las pruebas unitarias y de integración.

## 7. Aislamiento de Estado en Integración

- En las pruebas de integración del backend ejecutadas sobre Testcontainers y PostgreSQL, cada prueba debe ser estrictamente independiente y no presuponer un estado residual en la base de datos dejado por pruebas anteriores.
- Se debe utilizar la anotación `@Transactional` en las clases de prueba de Spring Boot para garantizar que cualquier mutación de datos se revierta (rollback) automáticamente al finalizar cada método de prueba. En escenarios donde `@Transactional` enmascare el comportamiento real (ej. pruebas de concurrencia), se debe implementar un script de limpieza (`@Sql(scripts = "classpath:truncate.sql")`) o utilizar utilidades de Testcontainers para reiniciar el estado de los datos.

## 8. Pruebas de Rendimiento y Benchmarking

- Debido a la naturaleza intensiva de los algoritmos de optimización ALNS e IPSO, el backend debe incluir un conjunto de pruebas de rendimiento (Microbenchmarks) utilizando la herramienta JMH (Java Microbenchmark Harness).
- Estas pruebas deben evaluar el tiempo de ejecución y el consumo de memoria (asignación de objetos) de los operadores de vecindario y la actualización del enjambre frente a instancias con grandes volúmenes de pedidos.
- Cualquier modificación en el código de optimización que incremente la complejidad temporal superando los umbrales de latencia definidos por el negocio (ej. > 5 segundos para 100 paradas) será considerada una regresión y deberá ser optimizada antes de su integración.

## 9. Referencias y recursos

Documentación de JUnit 5: <https://junit.org/junit5/>

Documentación de Mockito: <https://site.mockito.org/>

Documentación de Testcontainers: <https://testcontainers.com/>

Documentación de Vitest: <https://vitest.dev/>

Documentación de Mock Service Worker: <https://mswjs.io/>